Seminar 3 — Spring & Spring Boot

Cuprins

1. Spring si Spring Boot — notiuni generale
2. Configurare proiect
3. Entity - mapare JPA
4. DTO - Data Transfer Objects
5. Repository
6. Exception handling
7. Service
8. Controller & REST API
9. Testare cu Postman
10. Exercitii

> ⚠️ NOTA
>
> Fiecare student va realiza exemplele de mai jos pe calculatorul lui si de asemenea exercitiile de la final. Tot codul sursa pentru exercitiile individuale de la finalul documentului va fi incarcat pe online.ase.ro in sectiunea dedicata pana la finalul seminarului in vederea obtinerii unui punctaj.

1. Spring si Spring Boot — notiuni generale

1.1. Ce este Spring?

Spring: https://spring.io/

Spring Framework este cel mai popular framework Java pentru dezvoltarea aplicatiilor enterprise. Rezolva doua probleme fundamentale:

- **Inversion of Control (IoC)** — mai creezi obiectele cu `new`, Spring le creeaza si le gestioneaza
- **Dependency Injection (DI)** — Spring injecteaza automat dependintele unui obiect in loc sa le creezi manual

Fara Spring:

```
// Tu esti responsabil de creare si ciclul de viata al obiectelor
AccountRepository repo = new AccountRepository(new DataSource(...));
AccountService service = new AccountService(repo);
AccountController controller = new AccountController(service);
```

Cu Spring:

```
// Spring creeaza, injecteaza si gestioneaza totul
@RestController
public class AccountController {
    private final AccountService service; // injectat automat de Spring

    public AccountController(AccountService service) {
        this.service = service;
    }
}
```

1.2. Ce este Spring Boot?

Spring: https://spring.io/projects/spring-boot

Spring Boot este o extensie a Spring Framework care elimina configurarea manuala (XML, Java Config verbos). Aduce:

- **Auto-configuration** — detecteaza ce librarii ai in classpath si se configureaza singur
- **Embedded server** — Tomcat e inclus in JAR, nu mai ai nevoie de server extern
- **Starter dependencies** — grupuri de dependinte preconfigurate (`spring-boot-starter-web`, `spring-boot-starter-data-jpa`)
- **Production-ready** — health checks, metrics, logging out of the box

> In practica: Spring Boot = Spring + conventii sane + zero XML.

1.3. Concepte cheie

Spring Container (ApplicationContext)

Container-ul Spring e responsabil de crearea si gestionarea obiectelor numite **beans**. Un bean e orice obiect gestionat de Spring — service, repository, controller etc.

Bean lifecycle:

```
Creare → Injectare dependinte → Initializare (@PostConstruct) → Utilizare →
Distrugere (@PreDestroy)
```

Dependency Injection — tipuri

Tip	Cod	Recomandat
Constructor injection	`public Service(Repo r) { this.r = r; }`	Da
Field injection	`@Autowired private Repo r;`	Nu (greu de testat)
Setter injection	`@Autowired public void setRepo(Repo r)`	Situational

Constructor injection e preferata pentru ca face dependintele explicite si permite testarea usoara cu mock-uri.

1.4. Adnotari Spring Boot esentiale

Adnotari pentru definirea beans-urilor

Adnotare	Rol	Unde se pune
`@Component`	Bean generic gestionat de Spring	Orice clasa
`@Service`	Bean de tip logica de business	Clasa service
`@Repository`	Bean de tip acces la date, translateaza exceptii JPA	Clasa repository
`@RestController`	Bean HTTP controller, combina `@Controller` + `@ResponseBody`	Clasa controller
`@Configuration`	Clasa care defineste beans manual prin metode `@Bean`	Clasa de configurare

`@Service`, `@Repository`, `@RestController` sunt toate specializari ale `@Component` — Spring le trateaza la fel din perspectiva IoC, dar au semantica diferita si unele adauga comportament extra (`@Repository` translateaza exceptiile JPA in `DataAccessException`).

Adnotari pentru injectare

Adnotare	Rol
`@Autowired`	Spune Spring sa injecteze dependinta (optionala pe constructor in Spring 4.3+)
`@Qualifier("numeBean")`	Selecteaza un bean specific cand exista mai multe implementari
`@Value("${proprietate}")`	Injecteaza valoare din `application.properties`

Adnotari pentru configurare aplicatie

Adnotare	Rol
`@SpringBootApplication`	Punctul de intrare — combina `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`
`@ComponentScan`	Spune Spring unde sa caute beans (implicit pachetul curent si subpachetele)
`@EnableAutoConfiguration`	Activeaza auto-configurarea bazata pe classpath

Adnotari pentru REST

Adnotare	Rol
`@RequestMapping("/path")`	Mapeaza o clasa sau metoda la un URL
`@GetMapping("/path")`	Shortcut pentru `@RequestMapping(method = GET)`
`@PostMapping("/path")`	Shortcut pentru `@RequestMapping(method = POST)`
`@PutMapping("/path")`	Shortcut pentru `@RequestMapping(method = PUT)`
`@DeleteMapping("/path")`	Shortcut pentru `@RequestMapping(method = DELETE)`
`@PathVariable`	Extrage variabila din URL: `/accounts/{id}`
`@RequestParam`	Extrage query parameter: `/accounts?status=ACTIVE`
`@RequestBody`	Deserializeaza body-ul JSON in obiect Java
`@ResponseBody`	Serializeaza obiectul Java returnat in JSON (inclus in `@RestController`)

Adnotari pentru tranzactii

Adnotare	Rol
`@Transactional`	Marcheaza o metoda ca tranzactionala — Spring face commit/rollback automat

Adnotare	Rol
`@Transactional(readOnly = true)`	Optimizare pentru operatii de citire

1.5. Structura recomandata a proiectului

```
src/
└── main/
    ├── java/
    │   └── com.bdsa.bank/
    │       ├── BankApplication.java          ← punctul de intrare
(@SpringBootApplication)
    │       ├── controller/
    │       │   ├── AccountController.java
    │       │   └── TransactionController.java
    │       ├── service/
    │       │   ├── AccountService.java
    │       │   └── TransactionService.java
    │       ├── repository/
    │       │   ├── AccountRepository.java
    │       │   └── TransactionRepository.java
    │       ├── entity/
    │       │   ├── Account.java
    │       │   └── Transaction.java
    │       ├── dto/
    │       │   ├── AccountRequest.java
    │       │   ├── AccountResponse.java
    │       │   └── TransactionRequest.java
    │       └── exception/
    │           ├── GlobalExceptionHandler.java
    │           └── ResourceNotFoundException.java
    └── resources/
        └── application.properties
```

De ce aceasta separare?

- `entity/` — clasele mapate pe tabelele din DB (stiu despre baza de date)
- `dto/` — clasele folosite pentru request/response HTTP (stiu despre API)
- `repository/` — accesul la date (stiu despre entity)
- `service/` — logica de business (stiu despre repository si dto)
- `controller/` — gestioneaza request-urile HTTP (stiu despre service si dto)

Fiecare strat stie doar despre stratul imediat inferior, nu sare straturi.

1.6. Best practices

- **Nu injecta repository-ul direct in controller** — treci intotdeauna prin service
- **Logica de business sta in service, nu in controller** — controllerul doar "routeaza" request-urile
- **Foloseste DTO-uri** — nu expune entitatea JPA direct in API (expui structura interna a DB-ului)
- **Foloseste constructor injection** — nu `@Autowired` pe field
- **Marcheaza serviciile ca `@Transactional`** — nu repository-urile sau controller-ele
- **Returneaza `ResponseEntity<T>`** din controller — ai control deplin asupra status code-ului

2. Configurare proiect

2.1. Generare proiect cu Spring Initializr

Mergi la start.spring.io si configureaza:

- **Project:** Maven
- **Language:** Java
- **Spring Boot:** 3.5.x
- **Group:** `com.bdsa`
- **Artifact:** `bank`
- **Java:** 17

Dependencies de adaugat:

- `Spring Web` — pentru REST API
- `Spring Data JPA` — pentru Hibernate si JPA
- `Oracle Driver` — OJDBC pentru Oracle

Descarca arhiva, dezarhiveaz-o si deschid-o in IntelliJ.

2.2. pom.xml rezultat

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.3.4</version>
        <relativePath/>
    </parent>

    <groupId>com.bdsa</groupId>
    <artifactId>bank</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>bank</name>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <!-- REST API -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
```

```
        </dependency>

        <!-- JPA + Hibernate -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>

        <!-- Oracle JDBC Driver -->
        <dependency>
            <groupId>com.oracle.database.jdbc</groupId>
            <artifactId>ojdbc11</artifactId>
            <scope>runtime</scope>
        </dependency>

        <!-- Teste -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

2.3. application.properties

```
# Conexiune Oracle
spring.datasource.url=jdbc:oracle:thin:@//localhost:1521/XEPDB1
spring.datasource.username=tutorial_user
spring.datasource.password=parola_ta
spring.datasource.driver-class-name=oracle.jdbc.OracleDriver

# JPA / Hibernate
spring.jpa.database-platform=org.hibernate.dialect.OracleDialect
spring.jpa.hibernate.ddl-auto=none
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Port server (optional, default 8080)
server.port=8080
```

> ⚠️ **`ddl-auto=none`** — Hibernate NU va modifica tabelele existente. Tabelele le-ai creat deja manual in Seminar 2. Alte valori posibile: `create` (sterge si recreeaza), `update` (adauga coloane lipsa), `validate` (verifica structura, arunca exceptie daca nu corespunde). In productie folosesti intotdeauna `none` sau `validate`.

3. Entity — mapare JPA

Clasele entity sunt marcate cu `@Entity` si mapeaza direct pe tabelele din baza de date. Hibernate le foloseste pentru a genera si executa SQL-ul in fundal.

3.1. Account

```java
package com.bdsa.bank.entity;

import jakarta.persistence.*;
import java.time.LocalDate;
import java.util.List;

@Entity
@Table(name = "accounts")
public class Account {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "iban", nullable = false, unique = true, length = 34)
    private String iban;

    @Column(name = "owner_name", nullable = false, length = 100)
    private String ownerName;

    @Column(name = "balance", nullable = false)
    private Double balance;

    @Column(name = "currency", nullable = false, length = 3)
    private String currency;

    @Column(name = "status", nullable = false, length = 10)
    private String status;

    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDate createdAt;

    @OneToMany(mappedBy = "account", fetch = FetchType.LAZY)
    private List<Transaction> transactions;

    @PrePersist
    protected void onCreate() {
        if (this.createdAt == null) this.createdAt = LocalDate.now();
        if (this.status == null) this.status = "ACTIVE";
        if (this.balance == null) this.balance = 0.0;
    }

    // Constructori
    public Account() {}
```

```java
    public Account(String iban, String ownerName, Double balance, String currency)
{
        this.iban = iban;
        this.ownerName = ownerName;
        this.balance = balance;
        this.currency = currency;
    }

    // Getters & Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getIban() { return iban; }
    public void setIban(String iban) { this.iban = iban; }

    public String getOwnerName() { return ownerName; }
    public void setOwnerName(String ownerName) { this.ownerName = ownerName; }

    public Double getBalance() { return balance; }
    public void setBalance(Double balance) { this.balance = balance; }

    public String getCurrency() { return currency; }
    public void setCurrency(String currency) { this.currency = currency; }

    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }

    public LocalDate getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDate createdAt) { this.createdAt = createdAt; }

    public List<Transaction> getTransactions() { return transactions; }
    public void setTransactions(List<Transaction> transactions) {
this.transactions = transactions; }
}
```

Adnotari JPA explicate:

Adnotare	Rol
`@Entity`	Marcheaza clasa ca entitate JPA gestionata de Hibernate
`@Table(name = "accounts")`	Specifica numele tabelei in DB (implicit = numele clasei lowercase)
`@Id`	Marcheaza campul ca cheie primara
`@GeneratedValue(strategy = IDENTITY)`	ID generat de DB (corespunde cu `GENERATED ALWAYS AS IDENTITY` din Oracle)
`@Column(name = "owner_name")`	Mapeaza campul pe coloana cu alt nume decat campul Java

Adnotare	Rol
`@OneToMany(mappedBy = "account")`	Relatia 1:N catre tranzactii — `mappedBy` indica campul din Transaction care detine FK
`@PrePersist`	Metoda apelata de Hibernate inainte de primul INSERT

3.2. Transaction

```
package com.bdsa.bank.entity;

import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
@Table(name = "transactions")
public class Transaction {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "account_id", nullable = false)
    private Account account;

    @Column(name = "type", nullable = false, length = 6)
    private String type;

    @Column(name = "amount", nullable = false)
    private Double amount;

    @Column(name = "description", length = 255)
    private String description;

    @Column(name = "transaction_date", nullable = false, updatable = false)
    private LocalDate transactionDate;

    @PrePersist
    protected void onCreate() {
        if (this.transactionDate == null) this.transactionDate = LocalDate.now();
    }

    // Constructori
    public Transaction() {}

    public Transaction(Account account, String type, Double amount, String
description) {
        this.account = account;
        this.type = type;
        this.amount = amount;
        this.description = description;
```

```java
    }

    // Getters & Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public Account getAccount() { return account; }
    public void setAccount(Account account) { this.account = account; }

    public String getType() { return type; }
    public void setType(String type) { this.type = type; }

    public Double getAmount() { return amount; }
    public void setAmount(Double amount) { this.amount = amount; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description =
description; }

    public LocalDate getTransactionDate() { return transactionDate; }
    public void setTransactionDate(LocalDate transactionDate) {
this.transactionDate = transactionDate; }
}
```

> ⚠️ `FetchType.LAZY` **vs** `EAGER`
>
> `LAZY` — relatia se incarca din DB doar cand o accesezi explicit (`getTransactions()`). Recomandat pentru colectii si relatii `@ManyToOne` — evita query-uri inutile.
>
> `EAGER` — datele relationate sunt incarcate automat la fiecare query pe entitatea parinte. Poate cauza **N+1 problem**: daca ai 100 conturi si fiecare are tranzactii incarcate EAGER, Hibernate face 1 query pentru conturi + 100 query-uri pentru tranzactii = 101 query-uri in loc de 1 cu JOIN.

4. DTO — Data Transfer Objects

DTO-urile sunt clase simple folosite exclusiv pentru comunicarea prin API. **Nu expui entity-urile JPA direct** — acestea contin detalii interne ale DB-ului (relatii lazy, adnotari Hibernate) care pot cauza erori de serializare sau expun date sensibile.

4.1. AccountRequest (pentru POST/PUT)

```
package com.bdsa.bank.dto;

public class AccountRequest {
    private String iban;
    private String ownerName;
    private Double balance;
    private String currency;

    public AccountRequest() {}

    public String getIban() { return iban; }
    public void setIban(String iban) { this.iban = iban; }

    public String getOwnerName() { return ownerName; }
    public void setOwnerName(String ownerName) { this.ownerName = ownerName; }

    public Double getBalance() { return balance; }
    public void setBalance(Double balance) { this.balance = balance; }

    public String getCurrency() { return currency; }
    public void setCurrency(String currency) { this.currency = currency; }
}
```

4.2. AccountResponse (pentru GET)

```
package com.bdsa.bank.dto;

import com.bdsa.bank.entity.Account;

import java.time.LocalDate;

public class AccountResponse {
    private Long id;
    private String iban;
    private String ownerName;
    private Double balance;
    private String currency;
    private String status;
    private LocalDate createdAt;

    public AccountResponse() {}
```

```java
    // Constructor de mapare din entity
    public AccountResponse(Account account) {
        this.id = account.getId();
        this.iban = account.getIban();
        this.ownerName = account.getOwnerName();
        this.balance = account.getBalance();
        this.currency = account.getCurrency();
        this.status = account.getStatus();
        this.createdAt = account.getCreatedAt();
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getIban() { return iban; }
    public void setIban(String iban) { this.iban = iban; }

    public String getOwnerName() { return ownerName; }
    public void setOwnerName(String ownerName) { this.ownerName = ownerName; }

    public Double getBalance() { return balance; }
    public void setBalance(Double balance) { this.balance = balance; }

    public String getCurrency() { return currency; }
    public void setCurrency(String currency) { this.currency = currency; }

    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }

    public LocalDate getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDate createdAt) { this.createdAt = createdAt; }
}
```

4.3. TransactionRequest

```java
package com.bdsa.bank.dto;

public class TransactionRequest {
    private Long accountId;
    private String type;
    private Double amount;
    private String description;

    public TransactionRequest() {}

    public Long getAccountId() { return accountId; }
    public void setAccountId(Long accountId) { this.accountId = accountId; }

    public String getType() { return type; }
    public void setType(String type) { this.type = type; }
```

```
    public Double getAmount() { return amount; }
    public void setAmount(Double amount) { this.amount = amount; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description =
description; }
}
```

4.4. TransactionResponse

```
package com.bdsa.bank.dto;

import com.bdsa.bank.entity.Transaction;

import java.time.LocalDate;

public class TransactionResponse {
    private Long id;
    private Long accountId;
    private String type;
    private Double amount;
    private String description;
    private LocalDate transactionDate;

    public TransactionResponse() {}

    public TransactionResponse(com.bdsa.bank.entity.Transaction t) {
        this.id = t.getId();
        this.accountId = t.getAccount().getId();
        this.type = t.getType();
        this.amount = t.getAmount();
        this.description = t.getDescription();
        this.transactionDate = t.getTransactionDate();
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public Long getAccountId() { return accountId; }
    public void setAccountId(Long accountId) { this.accountId = accountId; }

    public String getType() { return type; }
    public void setType(String type) { this.type = type; }

    public Double getAmount() { return amount; }
    public void setAmount(Double amount) { this.amount = amount; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description =
description; }
```

```
    public LocalDate getTransactionDate() { return transactionDate; }
    public void setTransactionDate(LocalDate transactionDate) {
this.transactionDate = transactionDate; }
}
```

5. Repository

`JpaRepository<Entity, IdType>` ofera gratuit metodele CRUD de baza. Nu trebuie sa scrii implementari — Spring Data JPA le genereaza automat la runtime.

5.1. `AccountRepository`

```
package com.bdsa.bank.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;
import com.bdsa.bank.entity.Account;

import java.util.List;
import java.util.Optional;

@Repository
public interface AccountRepository extends JpaRepository<Account, Long> {

    // Spring Data genereaza SQL din numele metodei
    Optional<Account> findByIban(String iban);

    List<Account> findByStatus(String status);

    List<Account> findByOwnerNameContainingIgnoreCase(String name);

    // JPQL — query pe obiecte Java, nu pe tabele SQL
    @Query("SELECT a FROM Account a WHERE a.balance > :minBalance AND a.status =
'ACTIVE'")
    List<Account> findActiveWithBalanceAbove(@Param("minBalance") Double
minBalance);

    // Native SQL — pentru query-uri specifice Oracle
    @Query(value = "SELECT *\n" +
            "FROM (\n" +
            "    SELECT *\n" +
            "    FROM accounts\n" +
            "    ORDER BY balance DESC\n" +
            ")\n" +
            "WHERE ROWNUM <= :limit;", nativeQuery = true)
    List<Account> findTopByBalance(@Param("limit") int limit);
}
```

Metode existente din `JpaRepository`:

Metoda	Rol
`findAll()`	Returneaza toate inregistrarile
`findById(id)`	Returneaza `Optional<T>`
`save(entity)`	INSERT daca ID e null, UPDATE daca ID exista
`deleteById(id)`	Sterge dupa ID
`existsById(id)`	Returneaza `boolean`
`count()`	Numara inregistrarile

Query methods — conventii de naming:

Spring Data interpreteaza numele metodei si genereaza SQL:

```
findBy + CampJava + [Conditie]
  └── findByStatus            → WHERE status = ?
  └── findByStatusAndCurrency → WHERE status = ? AND currency = ?
  └── findByBalanceBetween    → WHERE balance BETWEEN ? AND ?
  └── findByOwnerNameContainingIgnoreCase → WHERE UPPER(owner_name) LIKE UPPER('%?
%')
```

5.2. TransactionRepository

```java
package com.bdsa.bank.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;
import com.bdsa.bank.entity.Transaction;

import java.time.LocalDate;
import java.util.List;

@Repository
public interface TransactionRepository extends JpaRepository<Transaction, Long> {

    List<Transaction> findByAccountId(Long accountId);

    List<Transaction> findByAccountIdAndType(Long accountId, String type);

    List<Transaction> findByTransactionDateBetween(LocalDate start, LocalDate
end);

    @Query("SELECT t FROM Transaction t WHERE t.account.id = :accountId ORDER BY
```

```
t.transactionDate DESC")
    List<Transaction> findByAccountIdOrderedByDate(@Param("accountId") Long
accountId);
}
```

6. Exception handling

Inainte de service si controller, definim exceptiile si handler-ul global — acesta va transforma exceptiile in raspunsuri HTTP corecte.

6.1. ResourceNotFoundException

```
package com.bdsa.bank.exception;

public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

6.2. InsufficientFundsException

```
package com.bdsa.bank.exception;

public class InsufficientFundsException extends RuntimeException {
    public InsufficientFundsException(String message) {
        super(message);
    }
}
```

6.3. GlobalExceptionHandler

```
package com.bdsa.bank.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<Map<String, Object>>
handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(Map.of(
                "timestamp", LocalDateTime.now(),
                "status", 404,
```

```
                "error", "Not Found",
                "message", ex.getMessage()
        ));
    }

    @ExceptionHandler(InsufficientFundsException.class)
    public ResponseEntity<Map<String, Object>>
handleInsufficientFunds(InsufficientFundsException ex) {
        return ResponseEntity.status(HttpStatus.UNPROCESSABLE_ENTITY).body(Map.of(
                "timestamp", LocalDateTime.now(),
                "status", 422,
                "error", "Unprocessable Entity",
                "message", ex.getMessage()
        ));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, Object>> handleGeneric(Exception ex) {
        return
ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(Map.of(
                "timestamp", LocalDateTime.now(),
                "status", 500,
                "error", "Internal Server Error",
                "message", "A aparut o eroare neasteptata."
        ));
    }
}
```

> `@RestControllerAdvice` intercepteaza exceptiile aruncate din orice controller si le transforma in `ResponseEntity` cu body JSON. Fara el, Spring ar returna pagina HTML de eroare Whitelabel implicita.

7. Service

Service-ul contine toata logica de business. Primeste DTO-uri, lucreaza cu entitati prin repository, returneaza DTO-uri.

7.1. AccountService

```
package com.bdsa.bank.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.bdsa.bank.dto.AccountRequest;
import com.bdsa.bank.dto.AccountResponse;
import com.bdsa.bank.entity.Account;
import com.bdsa.bank.exception.InsufficientFundsException;
import com.bdsa.bank.exception.ResourceNotFoundException;
import com.bdsa.bank.repository.AccountRepository;

import java.util.List;

@Service
@Transactional
public class AccountService {

    private final AccountRepository accountRepository;

    public AccountService(AccountRepository accountRepository) {
        this.accountRepository = accountRepository;
    }

    @Transactional(readOnly = true)
    public List<AccountResponse> findAll() {
        return accountRepository.findAll()
                .stream()
                .map(AccountResponse::new)
                .toList();
    }

    @Transactional(readOnly = true)
    public AccountResponse findById(Long id) {
        Account account = accountRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Contul cu ID " +
id + " nu exista."));
        return new AccountResponse(account);
    }

    @Transactional(readOnly = true)
    public AccountResponse findByIban(String iban) {
        Account account = accountRepository.findByIban(iban)
                .orElseThrow(() -> new ResourceNotFoundException("Contul cu IBAN "
+ iban + " nu exista."));
```

```java
        return new AccountResponse(account);
    }

    @Transactional(readOnly = true)
    public List<AccountResponse> findByStatus(String status) {
        return accountRepository.findByStatus(status)
                .stream()
                .map(AccountResponse::new)
                .toList();
    }

    public AccountResponse create(AccountRequest request) {
        Account account = new Account(
                request.getIban(),
                request.getOwnerName(),
                request.getBalance() != null ? request.getBalance() : 0.0,
                request.getCurrency() != null ? request.getCurrency() : "RON"
        );
        Account saved = accountRepository.save(account);
        return new AccountResponse(saved);
    }

    public AccountResponse update(Long id, AccountRequest request) {
        Account account = accountRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Contul cu ID " +
id + " nu exista."));

        if (request.getOwnerName() != null)
account.setOwnerName(request.getOwnerName());
        if (request.getCurrency() != null)
account.setCurrency(request.getCurrency());

        Account saved = accountRepository.save(account);
        return new AccountResponse(saved);
    }

    public void block(Long id) {
        Account account = accountRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Contul cu ID " +
id + " nu exista."));
        account.setStatus("BLOCKED");
        accountRepository.save(account);
    }

    public void delete(Long id) {
        Account account = accountRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Contul cu ID " +
id + " nu exista."));

        if (account.getBalance() > 0) {
            throw new InsufficientFundsException(
                "Contul nu poate fi sters. Soldul trebuie sa fie 0. Sold curent: "
+ account.getBalance()
            );
```

```
        }

        accountRepository.delete(account);
    }
}
```

7.2. TransactionService

```
package com.bdsa.bank.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.bdsa.bank.dto.TransactionRequest;
import com.bdsa.bank.dto.TransactionResponse;
import com.bdsa.bank.entity.Account;
import com.bdsa.bank.entity.Transaction;
import com.bdsa.bank.exception.InsufficientFundsException;
import com.bdsa.bank.exception.ResourceNotFoundException;
import com.bdsa.bank.repository.AccountRepository;
import com.bdsa.bank.repository.TransactionRepository;

import java.util.List;

@Service
@Transactional
public class TransactionService {

    private final TransactionRepository transactionRepository;
    private final AccountRepository accountRepository;

    public TransactionService(TransactionRepository transactionRepository,
                              AccountRepository accountRepository) {
        this.transactionRepository = transactionRepository;
        this.accountRepository = accountRepository;
    }

    @Transactional(readOnly = true)
    public List<TransactionResponse> findByAccountId(Long accountId) {
        if (!accountRepository.existsById(accountId)) {
            throw new ResourceNotFoundException("Contul cu ID " + accountId + " nu
exista.");
        }
        return transactionRepository.findByAccountIdOrderedByDate(accountId)
                .stream()
                .map(TransactionResponse::new)
                .toList();
    }

    public TransactionResponse create(TransactionRequest request) {
        Account account = accountRepository.findById(request.getAccountId())
                .orElseThrow(() -> new ResourceNotFoundException(
```

```
                        "Contul cu ID " + request.getAccountId() + " nu exista."));

        if ("DEBIT".equals(request.getType()) && account.getBalance() <
request.getAmount()) {
            throw new InsufficientFundsException(
                "Sold insuficient. Sold curent: " + account.getBalance() +
                ", suma ceruta: " + request.getAmount()
            );
        }

        // Actualizeaza soldul
        if ("CREDIT".equals(request.getType())) {
            account.setBalance(account.getBalance() + request.getAmount());
        } else {
            account.setBalance(account.getBalance() - request.getAmount());
        }
        accountRepository.save(account);

        // Salveaza tranzactia
        Transaction transaction = new Transaction(
                account,
                request.getType(),
                request.getAmount(),
                request.getDescription()
        );
        Transaction saved = transactionRepository.save(transaction);
        return new TransactionResponse(saved);
    }

    public void delete(Long id) {
        Transaction transaction = transactionRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException(
                    "Tranzactia cu ID " + id + " nu exista."));
        transactionRepository.delete(transaction);
    }
}
```

8. Controller si REST API

Conventii REST

Metoda HTTP	Actiune	Status success	Status eroare tipica
`GET`	Citire	`200 OK`	`404 Not Found`
`POST`	Creare	`201 Created`	`400 Bad Request`
`PUT`	Actualizare completa	`200 OK`	`404 Not Found`
`PATCH`	Actualizare partiala	`200 OK`	`404 Not Found`
`DELETE`	Stergere	`204 No Content`	`404 Not Found`

8.1. AccountController

```java
package com.bdsa.bank.controller;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import com.bdsa.bank.dto.AccountRequest;
import com.bdsa.bank.dto.AccountResponse;
import com.bdsa.bank.service.AccountService;

import java.util.List;

@RestController
@RequestMapping("/api/accounts")
public class AccountController {

    private final AccountService accountService;

    public AccountController(AccountService accountService) {
        this.accountService = accountService;
    }

    // GET /api/accounts
    @GetMapping
    public ResponseEntity<List<AccountResponse>> findAll() {
        return ResponseEntity.ok(accountService.findAll());
    }

    // GET /api/accounts/1
    @GetMapping("/{id}")
    public ResponseEntity<AccountResponse> findById(@PathVariable Long id) {
        return ResponseEntity.ok(accountService.findById(id));
    }

    // GET /api/accounts/iban/RO49AAAA...
```

```java
    @GetMapping("/iban/{iban}")
    public ResponseEntity<AccountResponse> findByIban(@PathVariable String iban) {
        return ResponseEntity.ok(accountService.findByIban(iban));
    }

    // GET /api/accounts?status=ACTIVE
    @GetMapping(params = "status")
    public ResponseEntity<List<AccountResponse>> findByStatus(@RequestParam String
status) {
        return ResponseEntity.ok(accountService.findByStatus(status));
    }

    // POST /api/accounts
    @PostMapping
    public ResponseEntity<AccountResponse> create(@RequestBody AccountRequest
request) {
        AccountResponse created = accountService.create(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }

    // PUT /api/accounts/1
    @PutMapping("/{id}")
    public ResponseEntity<AccountResponse> update(@PathVariable Long id,
                                                  @RequestBody AccountRequest
request) {
        return ResponseEntity.ok(accountService.update(id, request));
    }

    // PATCH /api/accounts/1/block
    @PatchMapping("/{id}/block")
    public ResponseEntity<Void> block(@PathVariable Long id) {
        accountService.block(id);
        return ResponseEntity.noContent().build();
    }

    // DELETE /api/accounts/1
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        accountService.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

8.2. TransactionController

```java
package com.bdsa.bank.controller;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import com.bdsa.bank.dto.TransactionRequest;
import com.bdsa.bank.dto.TransactionResponse;
import com.bdsa.bank.service.TransactionService;

import java.util.List;

@RestController
@RequestMapping("/api/transactions")
public class TransactionController {

    private final TransactionService transactionService;

    public TransactionController(TransactionService transactionService) {
        this.transactionService = transactionService;
    }

    // GET /api/transactions?accountId=1
    @GetMapping
    public ResponseEntity<List<TransactionResponse>> findByAccountId(@RequestParam
Long accountId) {
        return ResponseEntity.ok(transactionService.findByAccountId(accountId));
    }

    // POST /api/transactions
    @PostMapping
    public ResponseEntity<TransactionResponse> create(@RequestBody
TransactionRequest request) {
        TransactionResponse created = transactionService.create(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }

    // DELETE /api/transactions/1
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        transactionService.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

8.3. Punct de intrare — BankApplication

```java
package com.bdsa.bank;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BankApplication {
    public static void main(String[] args) {
        SpringApplication.run(BankApplication.class, args);
    }
}
```

Ruleaza aplicatia din IntelliJ (butonul Run) sau din terminal:

```
mvn spring-boot:run
```

Aplicatia porneste pe `http://localhost:8080`.

9. Testare cu Postman

9.1. Setup

- Deschide Postman si creeaza o noua **Collection** numita `BDSA Bank API`. In colectie poti salva toate request-urile organizate pe foldere (`Accounts`, `Transactions`).
- Seteaza o variabila de environment `baseUrl = http://localhost:8080` pentru a nu repeta URL-ul la fiecare request.

9.2. Accounts — exemple request/response

GET toate conturile

```
GET {{baseUrl}}/api/accounts
```

Response `200 OK`:

```
[
  {
    "id": 1,
    "iban": "RO49AAAA1B31007593840000",
    "ownerName": "Ion Popescu",
    "balance": 5000.00,
    "currency": "RON",
    "status": "ACTIVE",
    "createdAt": "2024-01-15"
  },
  {
    "id": 2,
    "iban": "RO49BBBB1B31007593840001",
    "ownerName": "Maria Ionescu",
    "balance": 12000.00,
    "currency": "RON",
    "status": "ACTIVE",
    "createdAt": "2024-01-15"
  }
]
```

GET cont dupa ID — gasit

```
GET {{baseUrl}}/api/accounts/1
```

Response `200 OK`:

```
{
  "id": 1,
  "iban": "RO49AAAA1B31007593840000",
  "ownerName": "Ion Popescu",
  "balance": 5000.00,
  "currency": "RON",
  "status": "ACTIVE",
  "createdAt": "2024-01-15"
}
```

GET cont dupa ID — negasit

```
GET {{baseUrl}}/api/accounts/999
```

Response `404 Not Found`:

```
{
  "timestamp": "2024-11-10T14:32:11.123",
  "status": 404,
  "error": "Not Found",
  "message": "Contul cu ID 999 nu exista."
}
```

GET conturi dupa status

```
GET {{baseUrl}}/api/accounts?status=ACTIVE
```

Response `200 OK` — lista de conturi active.

POST creare cont

```
POST {{baseUrl}}/api/accounts
Content-Type: application/json
```

Body:

```
{
  "iban": "RO49DDDD1B31007593840003",
  "ownerName": "Elena Marin",
  "balance": 1000.00,
```

```
  "currency": "RON"
}
```

Response `201 Created`:

```
{
  "id": 4,
  "iban": "RO49DDDD1B31007593840003",
  "ownerName": "Elena Marin",
  "balance": 1000.00,
  "currency": "RON",
  "status": "ACTIVE",
  "createdAt": "2024-11-10"
}
```

PUT actualizare cont

```
PUT {{baseUrl}}/api/accounts/4
Content-Type: application/json
```

Body:

```
{
  "ownerName": "Elena Marin-Popescu",
  "currency": "EUR"
}
```

Response `200 OK` — contul actualizat.

PATCH blocare cont

```
PATCH {{baseUrl}}/api/accounts/4/block
```

Response `204 No Content` — fara body, statusul conteaza.

DELETE stergere cont cu sold 0

```
DELETE {{baseUrl}}/api/accounts/3
```

Response `204 No Content`.

DELETE stergere cont cu sold > 0

```
DELETE {{baseUrl}}/api/accounts/1
```

Response `422 Unprocessable Entity`:

```
{
  "timestamp": "2024-11-10T14:35:00.456",
  "status": 422,
  "error": "Unprocessable Entity",
  "message": "Contul nu poate fi sters. Soldul trebuie sa fie 0. Sold curent:
5000.0"
}
```

9.3. Transactions — exemple request/response

GET tranzactii pentru un cont

```
GET {{baseUrl}}/api/transactions?accountId=1
```

Response `200 OK`:

```
[
  {
    "id": 2,
    "accountId": 1,
    "type": "DEBIT",
    "amount": 200.00,
    "description": "Factura curent",
    "transactionDate": "2024-01-15"
  },
  {
    "id": 1,
    "accountId": 1,
    "type": "CREDIT",
    "amount": 1000.00,
    "description": "Salariu",
    "transactionDate": "2024-01-15"
  }
]
```

POST tranzactie CREDIT

```
POST {{baseUrl}}/api/transactions
Content-Type: application/json
```

Body:

```
{
  "accountId": 1,
  "type": "CREDIT",
  "amount": 500.00,
  "description": "Bonus"
}
```

Response `201 Created`:

```
{
  "id": 5,
  "accountId": 1,
  "type": "CREDIT",
  "amount": 500.00,
  "description": "Bonus",
  "transactionDate": "2024-11-10"
}
```

POST tranzactie DEBIT cu sold insuficient

```
POST {{baseUrl}}/api/transactions
Content-Type: application/json
```

Body:

```
{
  "accountId": 3,
  "type": "DEBIT",
  "amount": 9999.00,
  "description": "Plata"
}
```

Response `422 Unprocessable Entity`:

```
{
  "timestamp": "2024-11-10T14:40:00.789",
  "status": 422,
  "error": "Unprocessable Entity",
  "message": "Sold insuficient. Sold curent: 0.0, suma ceruta: 9999.0"
}
```

DELETE tranzactie

```
DELETE {{baseUrl}}/api/transactions/2
```

Response `204 No Content`.

10. Exercitii

A. Endpoint nou pe AccountController

Adauga urmatoarele endpoint-uri in `AccountController` si `AccountService`:

1. `GET /api/accounts/top?limit=3` — returneaza primele `limit` conturi ordonate dupa sold DESC. Foloseste metoda `findTopByBalance` deja definita in repository.
2. `GET /api/accounts/{id}/balance` — returneaza doar soldul unui cont, ca numar simplu (nu obiect complet). Returneaza `404` daca contul nu exista.
3. `PATCH /api/accounts/{id}/unblock` — deblocheaza un cont (seteaza `status = 'ACTIVE'`). Returneaza `400 Bad Request` daca contul nu e in status `BLOCKED` (arunca o exceptie custom si prinde-o in `GlobalExceptionHandler`).

> HINT: Pentru endpoint-ul de balance, poti returna `ResponseEntity<Double>` direct din controller. Pentru unblock, verifica `account.getStatus().equals("BLOCKED")` in service inainte de a schimba statusul.

B. TransactionService extins

1. Adauga in `TransactionRepository` o metoda care calculeaza suma totala a tranzactiilor de tip CREDIT si DEBIT separat pentru un cont (tipul este ca parametru), folosind `@Query` cu `SUM` - `Double sumAmountByAccountAndType(accountId, type)`
2. Adauga in `TransactionService` metoda `Map<String, Double> getSummary(Long accountId)` care returneaza un map cu cheile `"totalCredit"` si `"totalDebit"`.
3. Expune endpoint-ul `GET /api/transactions/summary?accountId=1` in controller care returneaza acest map.

Exemplu response:

```
{
  "totalCredit": 6000.00,
  "totalDebit":  550.00
}
```

> HINT: `sumAmountByAccountAndType` din repository returneaza `Double` — poate fi `null` daca nu exista tranzactii de tipul respectiv. Foloseste `Optional.ofNullable(...).orElse(0.0)` pentru a evita NPE (Null Pointer Exception) sau poti modifica `@Query` astfel incat sa nu returneze nicioada `null`, ci 0 in cazlul acesta.

C. Transfer intre conturi

- Implementeaza endpoint-ul de transfer:

```
POST /api/accounts/transfer
```

Body:

```
{
  "fromAccountId": 1,
  "toAccountId": 2,
  "amount": 500.00,
  "description": "Chirie"
}
```

Creeaza un DTO `TransferRequest` cu campurile necesare. In `AccountService`, metoda `transfer(TransferRequest request)` trebuie sa:

1. Verifice ca ambele conturi exista (arunca `ResourceNotFoundException` daca nu).
2. Verifice ca ambele conturi sunt `ACTIVE` (arunca o exceptie custom `AccountNotActiveException` daca nu).
3. Verifice ca soldul contului sursa e suficient (arunca `InsufficientFundsException` daca nu).
4. Actualizeze soldurile si insereze doua tranzactii (DEBIT pe sursa, CREDIT pe destinatie).
5. Toata operatia sa fie `@Transactional` — daca orice pas esueaza, rollback complet.

> HINT: Metoda din service e deja `@Transactional` prin adnotarea de la nivel de clasa. Daca arunci o `RuntimeException` (sau subclasa ei) in interiorul metodei, Spring face rollback automat. Checked exceptions (`Exception`, `IOException`) nu declanseaza rollback implicit — adauga `@Transactional(rollbackFor = Exception.class)` daca vrei sa le incluzi.